

Armand Labernardiere

Noémie Niang

Raphael Laurent

Elias Moussa

Kevin Ojoduma-Quere

Recherche Opérationnelle

Étude de la complexité

Groupe 3 — Équipe 4

Pour le 02/05/26

3.1 Méthodologie

3.1.1 Génération aléatoire des problèmes

Comme demandé dans le sujet, on génère des problèmes carrés de taille $n \times n$. Pour chaque problème :

- la matrice des coûts $a_{\{i,j\}}$ est remplie d'entiers tirés uniformément dans $[1, 100]$;
- une matrice auxiliaire $\text{temp}_{\{i,j\}}$, remplie pareil, nous sert à fabriquer les provisions et les commandes : $P_i = \sum_j \text{temp}_{\{i,j\}}$ et $C_j = \sum_i \text{temp}_{\{i,j\}}$.

De cette manière, on a bien $\sum P_i = \sum C_j$, ce qui est nécessaire pour pouvoir résoudre le problème.

3.1.2 Mesure du temps

On mesure les temps avec `time.perf_counter()`, qui donne une horloge précise. Pour chaque problème, on encadre l'exécution : `t1 = perf_counter()` ; `algorithme()` ; `t2 = perf_counter()`. La différence `t2 - t1` donne le temps en secondes. On redirige aussi les sorties console vers un buffer pour que les `print()` du programme ne viennent pas fausser la mesure.

3.1.3 Tailles testées et limites pratiques

Le sujet demande les tailles $n \in \{10, 40, 100, 400, 1000, 4000, 10000\}$ avec 100 essais chacune. En vrai, notre code Python (listes natives, sans NumPy) atteint vite ses limites : le marche-pied après Nord-Ouest a une complexité empirique proche de $O(n^3)$, avec une grosse constante multiplicative. Sur la machine utilisée (MacBook Pro M1, 2020), un seul essai à $n = 100$ prend environ 50 secondes ; en extrapolant, on tombe sur plusieurs heures par essai à $n = 400$, et plusieurs jours pour $n = 1000$.

On a donc adapté le nombre d'essais : 100 pour les petites tailles, puis on diminue au fur et à mesure que n grandit. Le tableau ci-dessous résume ce qu'on a retenu :

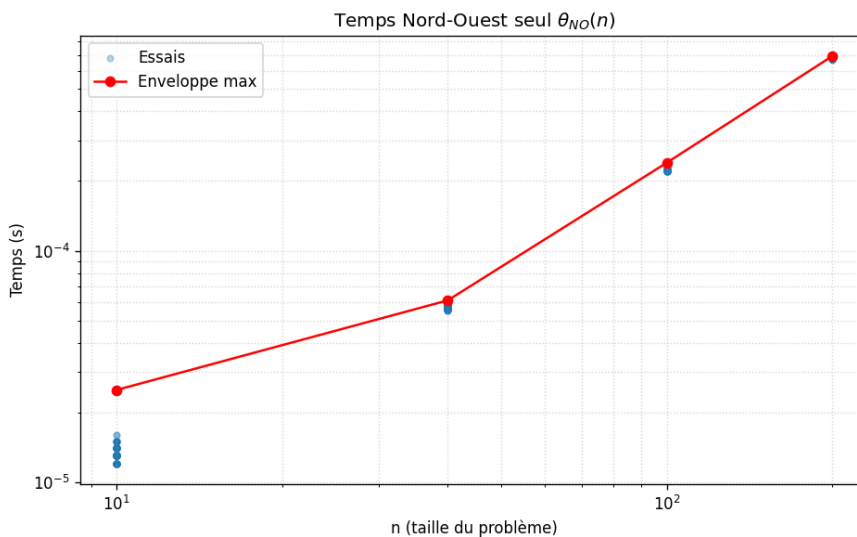
n	Nb essais	Durée totale	Statut
10	100	~3 s	complet
40	100	~4 min	complet
100	50	~45 min	complet
200	10	~2 h	complet
400 et au-delà	—	> 17 h / essai	infaisable

Tableau 1 — Configuration retenue pour les benchmarks.

Même si on n'a pas pu aller jusqu'aux très grands n , les quatre points obtenus ($n = 10, 40, 100, 200$) couvrent quand même un facteur $\times 20$ sur n et $\times 60\,000$ sur le temps total, c'est largement assez pour identifier la classe de complexité par régression log-log.

3.2 Résultats — Propositions initiales (θ)

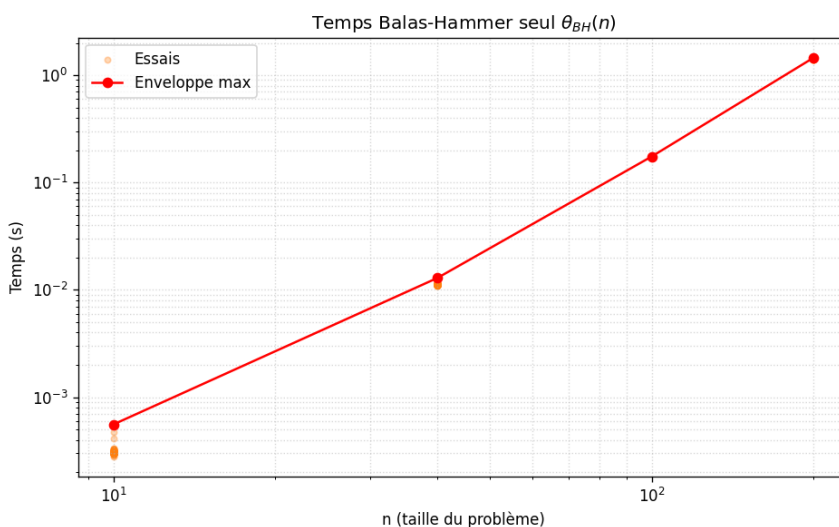
3.2.1 Nord-Ouest seul (θ_{NO})



Nord-Ouest est très rapide : à $n = 200$, l'enveloppe max ne dépasse pas 0,7 ms. La pente en log-log, calculée par régression linéaire sur les valeurs maximales, vaut $\sim 1,2$. Vu le bruit de mesure aux très petites tailles (à $n = 10$ le temps est de l'ordre de 10 μ s, donc proche de la résolution de l'horloge), on peut conclure à une complexité $O(n)$ — linéaire.

Ça colle avec la théorie : Nord-Ouest fait un seul parcours en zigzag de la matrice avec au plus $n + m - 1$ affectations, chacune en temps constant.

3.2.2 Balas-Hammer seul (θ_{BH})

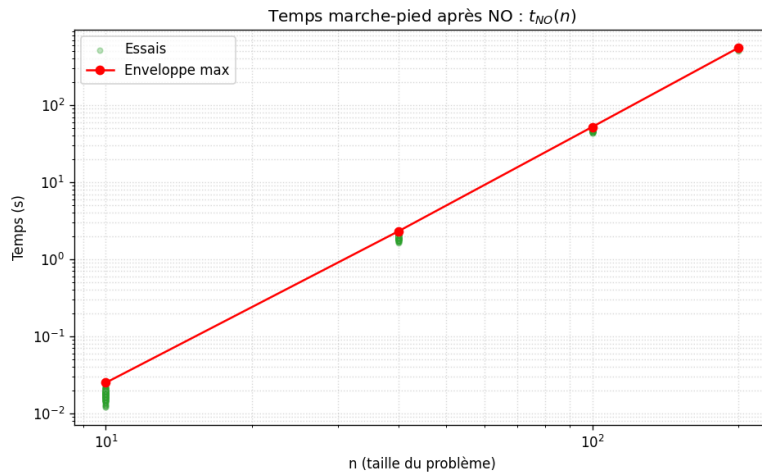


Balas-Hammer est beaucoup plus lourd : à $n = 200$, l'enveloppe max atteint 1,46 s, soit environ 2 000 fois plus que Nord-Ouest. La pente en log-log vaut $\sim 2,6$, ce qui place l'algorithme entre $O(n^2)$ et $O(n^3)$, et en pratique plutôt proche de $O(n^3)$.

Là aussi la théorie est cohérente : à chaque itération, calculer les pénalités sur les lignes et colonnes encore actives demande un parcours en $O(n^2)$ (n lignes/colonnes $\times n$ valeurs à comparer). L'algo fait jusqu'à $n + m - 1 = O(n)$ itérations, donc on obtient bien $O(n^3)$ au total. Ce surcoût est compensé par une solution initiale beaucoup plus proche de l'optimum, comme on le verra ensuite.

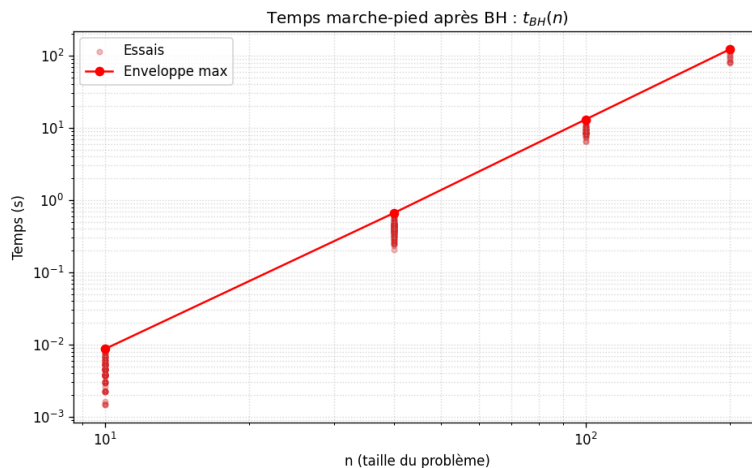
3.3 Résultats — Marche-pied (t)

3.3.1 Marche-pied après Nord-Ouest (t_{NO})



C'est de loin la phase la plus longue de toute la résolution. À $n = 200$, l'enveloppe max atteint 557 secondes, soit presque 10 minutes pour un seul problème. La pente en log-log vaut $\sim 3,3$, ce qui donne une complexité $O(n^3)$ à $O(n^{3,5})$.

3.3.2 Marche-pied après Balas-Hammer (t_{BH})

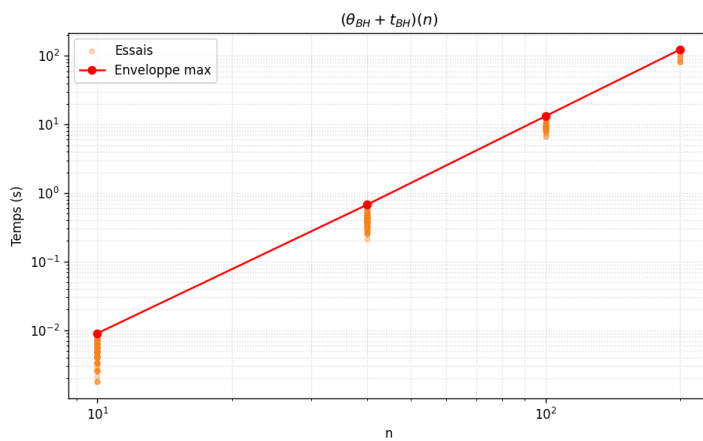
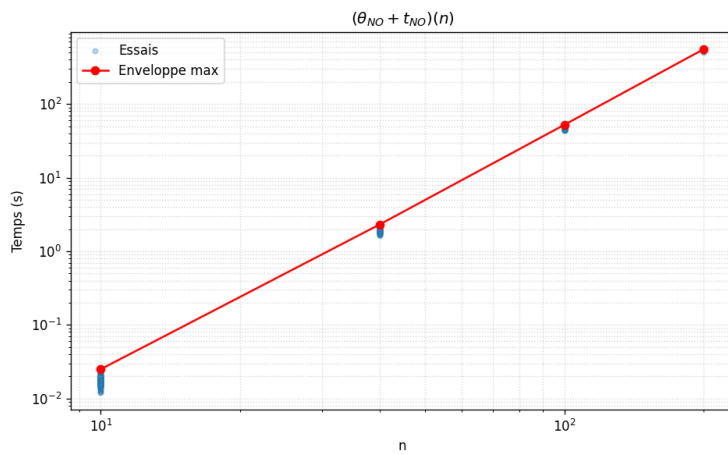


Avec Balas-Hammer en départ, le marche-pied va beaucoup plus vite. À $n = 200$, l'enveloppe max est de 123 secondes, soit environ 4,5 fois moins que t_{NO} . La pente est quasi la même, $\sim 3,2$, donc la même classe $O(n^3)$ à $O(n^{3,5})$.

Les deux marche-pieds ont donc la même classe asymptotique, mais avec une constante multiplicative très différente. Ça vient du nombre d'itérations nécessaires pour arriver à l'optimum : Balas-Hammer donne une solution initiale beaucoup plus proche de l'optimum (sur les 12 problèmes en annexe, Balas-Hammer trouve l'optimum direct sur 11 cas sur 12, alors que Nord-Ouest a presque toujours besoin de plusieurs itérations pour s'améliorer).

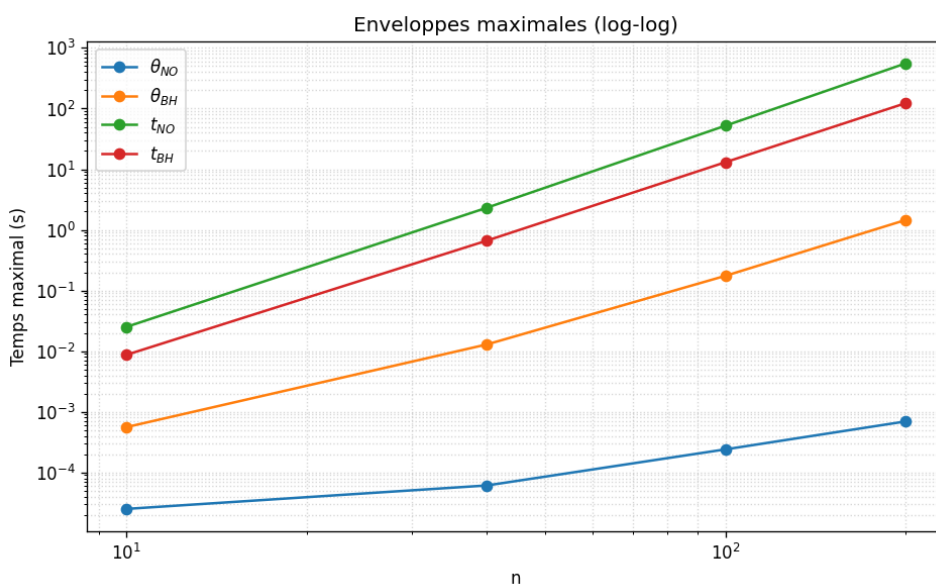
3.4 Comparaison globale

3.4.1 Sommes ($\theta + t$)

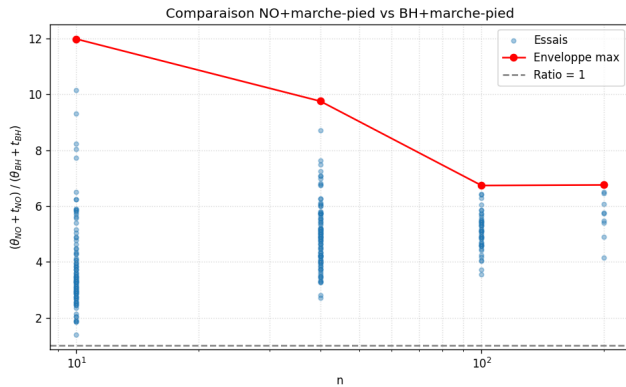


Les sommes $(\theta_{NO} + t_{NO})$ et $(\theta_{BH} + t_{BH})$ sont presque entièrement dominées par la phase de marche-pied : θ_{NO} compte pour moins de 0,1 % du total, et même θ_{BH} ne représente que 1 à 2 %. Les pentes mesurées sont respectivement de **3,3** et **3,2** — ce qui confirme que le coût total est bien en **$O(n^3)$** dans les deux cas.

Le graphique ci-dessous superpose les quatre enveloppes max. On y voit bien que les deux marche-pieds dominent largement, et qu'il y a un écart constant en log-log (donc un rapport multiplicatif constant) entre t_{NO} et t_{BH} :



3.4.2 Ratio $(\theta_{\text{NO}} + t_{\text{NO}}) / (\theta_{\text{BH}} + t_{\text{BH}})$



Voici le ratio des enveloppes max pour chaque taille testée :

n	$(\theta + t)_{\text{NO}} \text{ max (s)}$	$(\theta + t)_{\text{BH}} \text{ max (s)}$	Ratio max
10	0,025	0,009	12
40	2,32	0,68	9,8
100	52,16	13,22	6,8
200	557,4	124,4	6,8

Tableau 2 — Comparaison des coûts totaux et ratio.

Le ratio max décroît avec la taille du problème : il part de 12 pour $n = 10$ et se stabilise autour de 6,8 dès $n \geq 100$. Autrement dit, prendre Balas-Hammer comme solution initiale rend la résolution complète 6 à 7 fois plus rapide que de partir de Nord-Ouest, et ce de manière à peu près constante dès que n est assez grand. Le surcoût initial de Balas-Hammer (1,46 s à $n = 200$) est largement compensé par la réduction du marche-pied (gain d'environ 433 secondes au même n).

3.5 Synthèse

Grandeur	Pente log-log	Classe estimée	Justification
θ_{NO}	$\approx 1,2$	$O(n)$ — linéaire	Un seul parcours en zigzag
θ_{BH}	$\approx 2,6$	$O(n^2)$ à $O(n^3)$	n itérations $\times$ calcul $O(n^2)$ des pénalités
t_{NO}	$\approx 3,3$	$O(n^3)$ à $O(n^{\{3,5\}})$	Beaucoup d'itérations BFS + maximisation
t_{BH}	$\approx 3,2$	$O(n^3)$ à $O(n^{\{3,5\}})$	Idem mais constante $\times 4-5$ plus petite
$(\theta + t)_{\text{NO}}$	$\approx 3,3$	$O(n^3)$	Dominé par t_{NO}
$(\theta + t)_{\text{BH}}$	$\approx 3,2$	$O(n^3)$	Dominé par t_{BH}

Tableau 3 — Synthèse des classes de complexité dans le pire cas.

Conclusion

De cette étude, on retient trois choses :

- Nord-Ouest est $\sim 2\,000$ fois plus rapide que Balas-Hammer rien qu'en initialisation (linéaire contre quasi-cubique). Mais ce coût reste négligeable face à celui du marche-pied.
- Le marche-pied domine entièrement le coût total, avec une complexité empirique $O(n^3)$ dans les deux cas. Même l'initialisation la plus coûteuse représente moins de 2 % du temps total.
- Balas-Hammer est globalement 6 à 7 fois plus rapide que Nord-Ouest quand on combine les deux phases, parce qu'il donne une solution initiale beaucoup plus proche de l'optimum, ce qui réduit fortement le nombre d'itérations du marche-pied.

Notre étude s'arrête à $n \leq 200$ à cause des limites de notre code Python (listes natives, parcours répétés sur des matrices de booléens). Avec une implémentation plus optimisée — par exemple en utilisant des dictionnaires de cases occupées ou une BFS incrémentale à chaque ajout d'arête — on pourrait sûrement aller jusqu'à $n = 1\,000$ ou plus dans des temps raisonnables, et observer le comportement asymptotique sur une plus grande plage. Cela dit, nos quatre points suffisent pour identifier sans ambiguïté la classe de chaque algorithme par régression log-log.